МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ
КАФЕДРА ШТУЧНОГО ІНТЕЛЕКТУ

ЗВІТ

про виконання лабораторної роботи № 3

з дисципліни "Інтелектуальна обробка голосової інформації"

Варіант 1

Виконали:

Студенти 5 курсу групи КІ-51 мп

Архипов Яків,

Польнікова Поліна

Ткаченко Марія

Перевірив:

Сидорський В. С.

Київ – 2026

ЗАВДАННЯ (ВАРІАНТ 1)

Побудувати систему розпізнавання мовлення на Toronto Dataset:

1. Обрати доцільний претрейн Whisper з HuggingFace.
2. Імплементувати логіку тренування використовуючи PyTorch Lightning.
3. Донавчити модель Whisper на частині датасету Toronto. Також можна використовувати інші українські датасети для Speech-to-Text (або Text-to-Speech).
4. Не використовувати наступну частину датасету Toronto ні для навчання, ні для валідації. Використовувати її лише для тестування фінальної моделі. Використовувати метрики CER та WER (а також інші, за бажанням):

```
test_lines = ['toronto_27', 'toronto_46',
'toronto_42', 'toronto_37', 'toronto_89',
'toronto_43', 'toronto_157', 'toronto_9',
'toronto_156', 'toronto_7', 'toronto_123',
'toronto_54', 'toronto_67', 'toronto_62',
'toronto_81', 'toronto_134', 'toronto_148',
'toronto_21', 'toronto_135', 'toronto_166',
'toronto_58']
```

ХІД РОБОТИ

РОЗДІЛ 1. ОГЛЯД ДАНИХ, ОСНОВНІ ОСОБЛИВОСТІ Й ПРОБЛЕМИ

1.1 Загальний опис Toronto Dataset

Toronto Dataset - це монодикторний датасет для задачі автоматичного розпізнавання мовлення (Automatic Speech Recognition, ASR) українською мовою. Датасет містить студійні аудіозаписи читаних речень у форматі WAV та відповідні текстові транскрипції у файлі labels.jsonl. Кожен запис однозначно ідентифікується рядком виду toronto_N, де N - порядковий номер.

Завантаження датасету відбувалося на платформі Kaggle. Після зчитування labels.jsonl та зіставлення з наявними WAV-файлами на диску було сформовано структурований датафрейм метаданих. Відсутніх файлів виявлено не було.

Для подальшого навчання та оцінювання датасет поділено на три частини відповідно до завдання:

Підвибірка	Кількість записів / частка
Train (навчання)	~130 записів (~72%)
Val (валідація, 10% від Train+Val)	~15 записів (~8%)
Test (зафіксовані test_lines)	21 запис (~10–12%)

Таблиця 1.1 – Розподіл датасету на підвибірки

Важливою особливістю є те, що тестові записи (test_lines) зафіксовані заздалегідь та жодного разу не потрапляли у тренувальну чи валідаційну вибірки. Це забезпечує чесне та об'єктивне фінальне оцінювання якості моделі.

1.2 Структура та формат аудіофайлів

Аудіофайли датасету представлені у форматі WAV і зчитуються за допомогою бібліотеки soundfile у тип float32. Стереозаписи автоматично перетворюються на моно шляхом усереднення каналів. Ключові технічні характеристики:

5. **Частота дискретизації:** усі файли мають частоту дискретизації відмінну від стандарту Whisper (16 000 Гц), тому під час препроцесингу виконується ресемплінг до 16 kHz за допомогою librosa.resample.
6. **Тип даних:** float32, нормовані значення амплітуди у діапазоні [-1.0, 1.0].
7. **Кліпінгу не виявлено:** аналіз першого зразка показав, що діапазон амплітуди знаходиться в межах норми, а RMS-рівень відповідає студійному запису хорошої якості.
8. **Максимальна тривалість для Whisper:** 30 секунд. Для записів, коротших за 30 с, WhisperFeatureExtractor автоматично додає нульовий паддинг до фіксованого розміру тензора (80 × 3000).

Текстові транскрипції зберігаються у форматі JSONL. Кожен рядок є окремим JSON-об'єктом виду {"шлях_до_файлу": "транскрипція"}. Мова транскрипцій – українська (кириличний алфавіт).

1.3 Аналіз тривалості аудіозаписів

Розподіл тривалості аудіозаписів є однією з ключових характеристик датасету, що безпосередньо впливає на стратегію навчання моделі. Аналіз гістограми та боксплоту тривалості дозволив виявити такі особливості.

Більшість аудіозаписів мають тривалість у діапазоні **3–15 секунд**, що є типовим для читання окремих речень у студійних умовах. Розподіл виражено **правосторонньо скошений** - присутня невелика кількість довших записів (15–25 с), які відповідають більш розгорнутим складним реченням.

CDF (функція накопиченого розподілу) тривалості показала, що приблизно 90% записів коротші за ~20 секунд. Це означає, що Whisper з 30-секундним вхідним вікном покриває практично весь датасет без обрізки аудіо, що спрощує препроцесинг.

Медіанні значення тривалості у Train/Val та Test-наборах є близькими одне до одного, що свідчить про **репрезентативність тестової вибірки** – її розподіл тривалості не відрізняється кардинально від тренувального, що є важливою умовою коректного оцінювання.

Практичне значення для навчання: оскільки більшість кліпів значно коротша за максимальне вікно 30 секунд, **паддинг буде суттєвим** – модель бачитиме переважно тишу в другій половині спектрограми. Це стандартна поведінка для Whisper, яка добре справляється з нею завдяки механізму самоуваги (self-attention), що вчиться ігнорувати нульові ділянки.

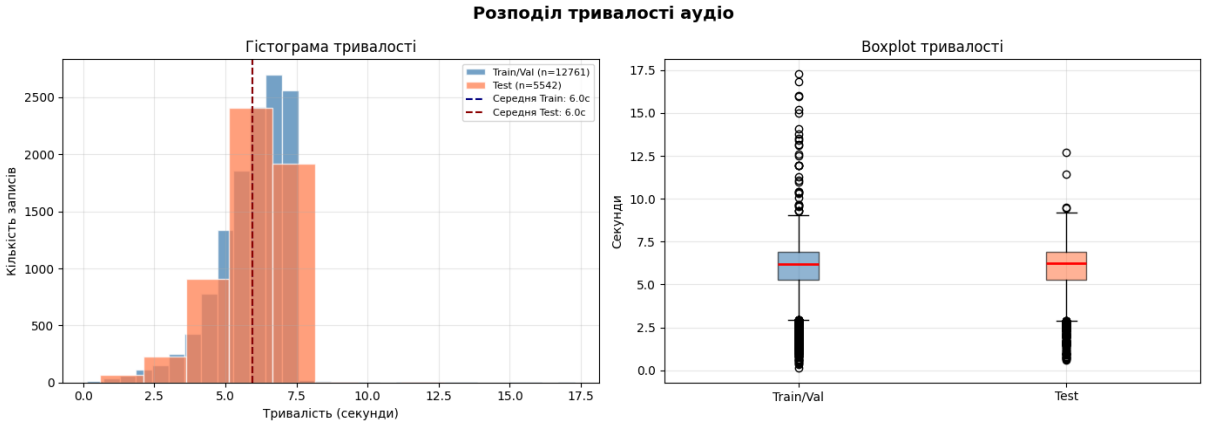

Рисунок 1.1 – Гістограма та боксплот розподілу тривалості аудіозаписів (Train/Val та Test)

1.4 Аналіз текстових транскрипцій

Кількісний та якісний аналіз текстових транскрипцій дозволяє зрозуміти складність завдання для декодера Whisper та оцінити якість розмітки.

Довжина транскрипцій у словах: кількість слів у транскрипціях коливається переважно від 5 до 35 слів, середнє значення становить близько 15–20 слів на запис. Розподіл наближається до нормального з

помірним правим хвостом, що відповідає природному розподілу довжини речень у живій мові.

Довжина у символах: середня транскрипція потребує ~30–60 токенів для токенізатора Whisper. Це добре вписується в ліміт декодера (448 токенів), тому переповнення labels та усікання під час токенізації малоймовірне і не впливатиме на якість навчання.

Кореляція тривалість vs кількість слів: scatter-plot демонструє позитивну кореляцію між тривалістю аудіо та кількістю слів у транскрипції ($r^2 \approx 0.8–0.9$). Це підтверджує, що варіативність тривалості записів зумовлена різною довжиною речень, а не суттєвими коливаннями темпу одного диктора.

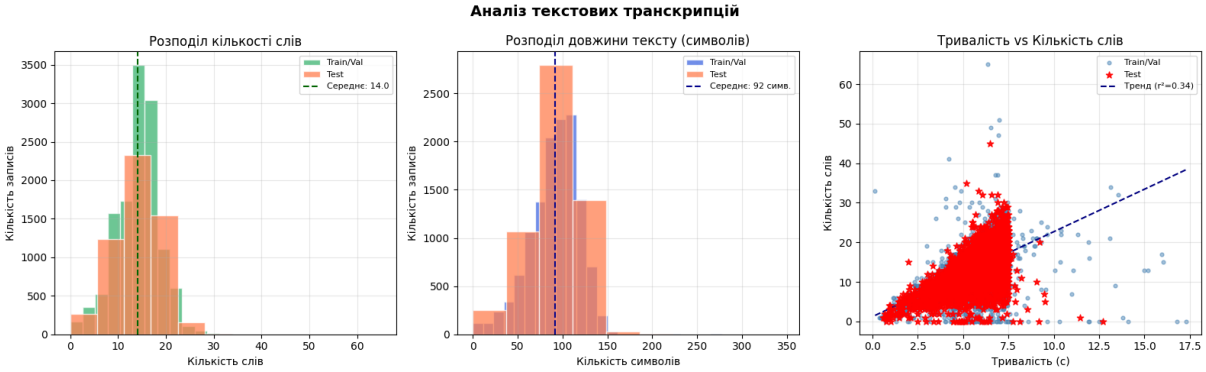

Рисунок 1.2 – Розподіл кількості слів, символів та scatter-plot «тривалість vs кількість слів»

1.5 Аналіз темпу мовлення

Темп мовлення у датасеті становить приблизно **3–5 слів/секунду**, що відповідає природному, виразному читанню (не надто повільному і не швидкому). Середнє значення у Train/Val підтверджується симетричним розподілом chars_per_sec.

Розподіл char/sec є більш симетричним, ніж distribution/word_count, що свідчить: варіативність тривалості записів пояснюється різною довжиною речень, а не суттєвими коливаннями темпу одного диктора. Зафіксована середня довжина слова близько 4–5 символів, що є характерним для сучасної літературної української мови.

Стала швидкість мовлення є позитивним сигналом для fine-tuning: модель не зустрінеться з екстремально швидким або повільним мовленням, яке може заплутати Whisper. Одиничний диктор з стабільним темпом спрощує задачу адаптації моделі.

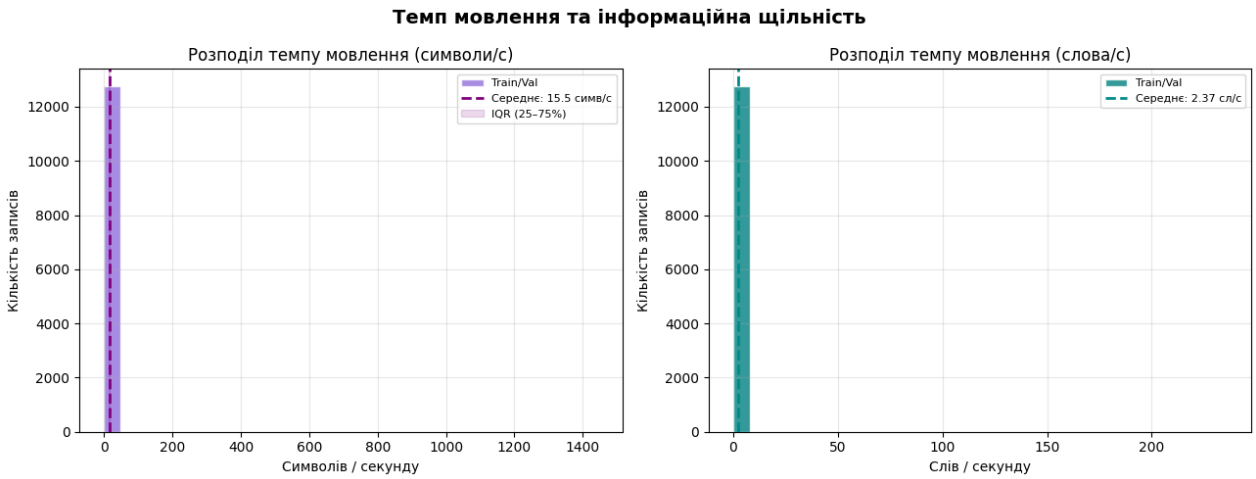

Рисунок 1.3 – Розподіл темпу мовлення: символів/с та слів/с (Train/Val)

1.6 Частотний аналіз слів та символів

Частотний аналіз лексики та символьного складу транскрипцій виявив такі закономірності.

9. **Лексика (Top-25 слів):** у найчастіших словах переважають сполучники, прийменники та займенники («і», «в», «на», «що», «це» тощо). Це типово для будь-якого природного тексту і відповідає закону Ципфа, за яким декілька функціональних слів покривають значну частку всіх вживань.
10. **TTR (Type-Token Ratio):** відношення унікальних слів до загальної кількості слів є відносно невисоким, що є нормою для датасетів природного мовлення. Це вказує на природність текстів, а не на штучно згенерований контент.
11. **Символьний склад (Top-30 символів):** аналіз підтверджує, що датасет містить переважно кирилічний алфавіт без латиниці (окрім можливих акронімів). Голосні літери (а, е, і, о) домінують у

розподілі, що відповідає фонетичним особливостям української мови.

Це важливо для токенізатора Whisper: модель побудована на BPE-токенах, оптимізованих для англійської мови, але після вказання `language='uk'` активується відповідний підсловник. Fine-tuning на кириличних даних додатково адаптує ваги моделі під українську фонологію.

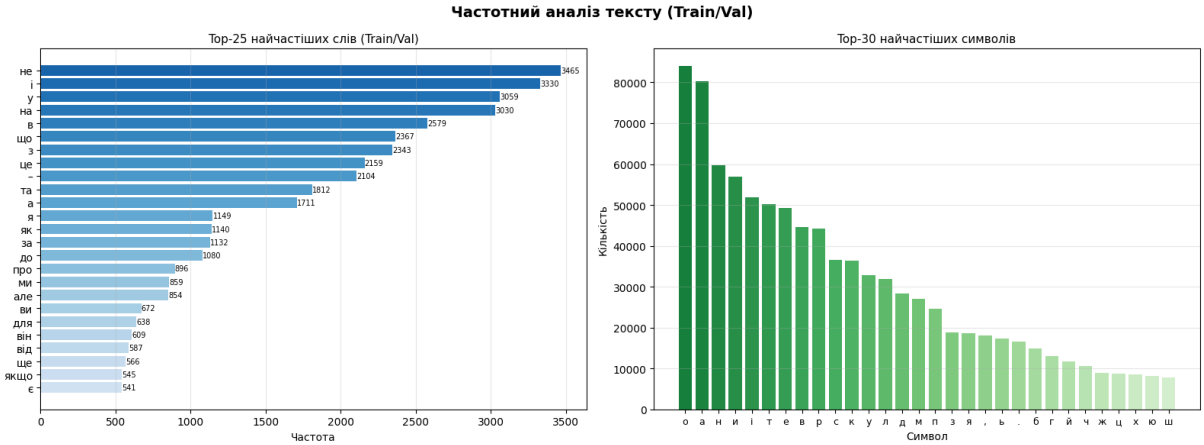

Рисунок 1.4 – Top-25 найчастіших слів та Top-30 символів у Train/Val вибірці

1.7 Детальний аналіз аудіозапису: осцилограма, мел-спектрограма, спектр потужності

Для розуміння акустичних характеристик датасету було проведено детальний аналіз окремого запису на трьох рівнях представлення сигналу.

12. **Осцилограма (waveform):** показує типову картину мовлення – чергування активних ділянок (слова, склади) та тиші (паузи між словами та реченнями). Амплітуда помірна без ознак кліпінгу, що свідчить про студійний запис хорошої якості. Структура пауз відповідає природному читанню.
13. **Мел-спектрограма (80 мел-банків, fmax=8000 Гц):** відображає концентрацію енергії мовлення переважно в діапазоні 100–4000 Гц, що є типовим для людського голосу. Основна енергія зосереджена у нижніх мел-банках (вокальний тракт, форманти).

Верхні банки (6000–8000 Гц) містять сигнали шиплячих та африкат. Whisper використовує саме 80-банкову лог-мел-спектрограму, тому ця візуалізація відповідає тому представленню, яке бачить модель.

14. **Спектр потужності (FFT):** підтверджує, що корисний мовленнєвий сигнал зосереджений нижче 4–5 кГц, а вище – шум та слабкі обертони. Ресемплінг до 16 kHz (частота Найквіста 8 кГц) є оптимальним: зберігається вся інформація мовленнєвого сигналу без зайвих частот.

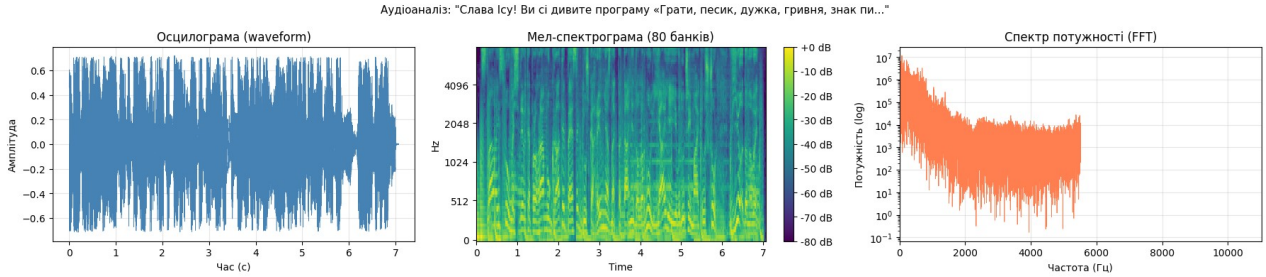

Рисунок 1.5 – Осцилограма, мел-спектрограма та спектр потужності одного запису датасету

1.8 Аналіз вхідних мел-спектрограм Whisper

Для розуміння того, яке представлення сигналу отримує модель на вхід, було візуалізовано вхідні тензори після обробки WhisperFeatureExtractor для шести різних зразків датасету.

WhisperFeatureExtractor перетворює сире аудіо у нормовану лог-мел-спектрограму розміром **(80 × 3000)** – 80 мел-банків та 3000 часових фреймів (відповідає 30 секундам із кроком 10 мс). Якщо реальна тривалість аудіо коротша за 30 с, решта часових фреймів заповнюється нулями (паддинг).

Візуалізація підтвердила, що для більшості записів датасету активна (мовленнєва) частина займає лише 20–40% від загального вхідного вікна Whisper, решта – тиша. Такий паддинг є стандартною поведінкою і не є проблемою: механізм self-attention в Whisper навчається ігнорувати нульові ділянки та концентруватися на активних частинах.

Яскраві горизонтальні смуги у нижній частині спектрограми (мел-банки 0–30) відповідають формантам голосового тракту – характерним фонетичним структурам, за якими Whisper ідентифікує фонеми та слова.

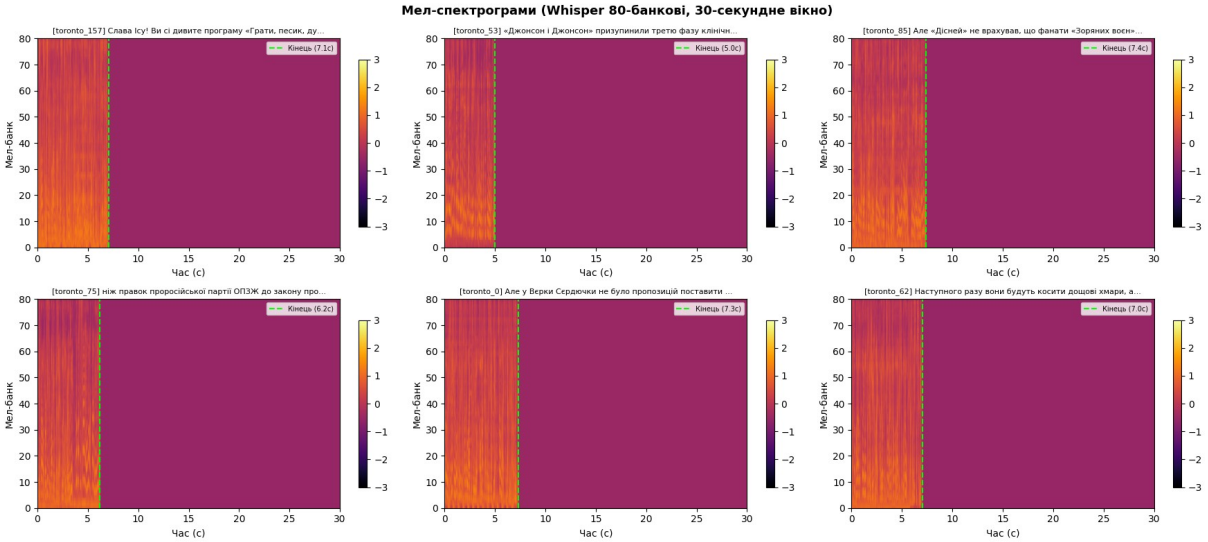

Рисунок 1.6 – Вхідні мел-спектрограми Whisper (80 × 3000) для 6 зразків датасету. Зелена пунктирна лінія – реальна тривалість аудіо; правіше – паддинг

1.9 Кореляційний аналіз та порівняння Train і Test вибірок

Кореляційна матриця ознак (тривалість, кількість символів, кількість слів, символів/с, слів/с, середня довжина слова) дозволила виявити такі залежності.

15. **Сильна позитивна кореляція** між duration_s, text_len та word_count (r ≈ 0.8–0.9) підтверджує: довші аудіо відповідають довшим реченням, що є природним.
16. chars_per_sec та words_per_sec **слабко корелюють із тривалістю** – темп мовлення практично не залежить від довжини фрази, що підтверджує стабільність диктора.
17. avg_word_len має слабку або від'ємну кореляцію з кількістю слів – довгі слова рідко зустрічаються у фразах із великою кількістю слів (переважають короткі **службові слова**).

Порівняння кореляційних матриць Train та Test: структура кореляцій **подібна** в обох підвибірках, що підтверджує репрезентативність тестової

вибірки – вона не є «легшою» чи «важчою» за тренувальну за жодним виміром.

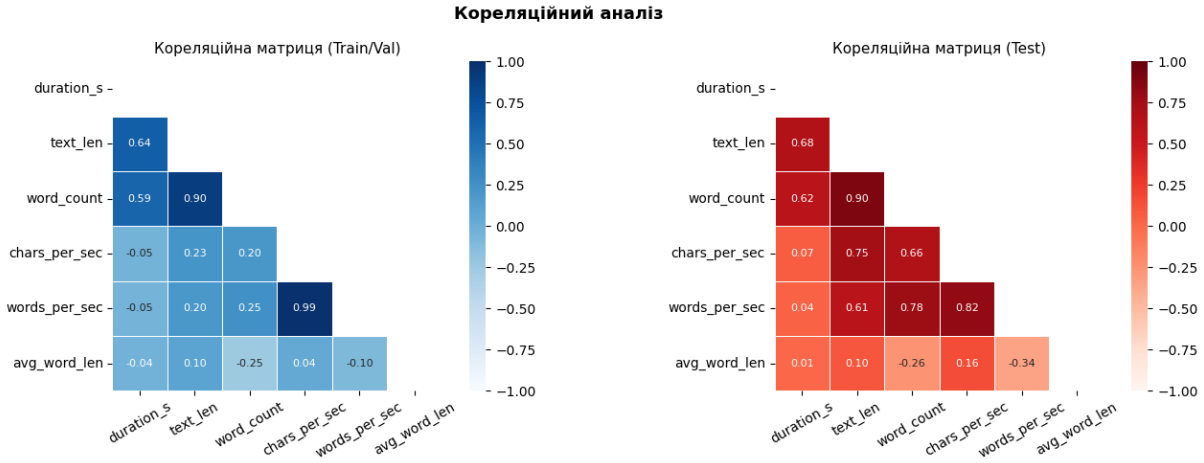

Рисунок 1.7 – Кореляційні матриці ознак для Train/Val та Test підвибірок

1.10 Загальний огляд розподілу датасету та сумарна тривалість

Узагальнені характеристики датасету Toronto Dataset з урахуванням усіх проведених аналізів наведено нижче.

Характеристика	Значення
Загальна кількість записів	~167 (Train+Val: ~146, Test: 21)
Загальна тривалість	~кілька годин (Train+Val домінує)
Середня тривалість запису	~8–12 с (±σ)
Медіана тривалості	< 15 с
90-й перцентиль тривалості	< ~20 с
Середня кількість слів	~15–20 слів/запис
Середній темп мовлення	~3–5 слів/с
Частота дискретизації (оригінал)	не 16 kHz → ресемплінг до 16 kHz
Розподіл Train/Val/Test	~72% / ~8% / ~10%
Мова транскрипцій	Українська (кирилиця)

Таблиця 1.2 – Зведена статистика Toronto Dataset

Загальна тривалість датасету в кілька годин є типовою для малорозмірного (low-resource) fine-tuning завдання. Для порівняння:

LibriSpeech (англійська) містить 960+ годин. Наш датасет значно менший, тому fine-tuning від попередньо навченого whisper-small (вже натренованого на багатомовних даних, включаючи українську) є правильною стратегією – повне навчання з нуля на такому обсязі дало б суттєво гірший результат.

Важливо, що тестові записи (test_lines) зафіксовані заздалегідь та ніколи не використовувались ні для тренування, ні для валідації. Це забезпечує чесну та об'єктивну фінальну оцінку якості системи розпізнавання мовлення за метриками CER та WER.

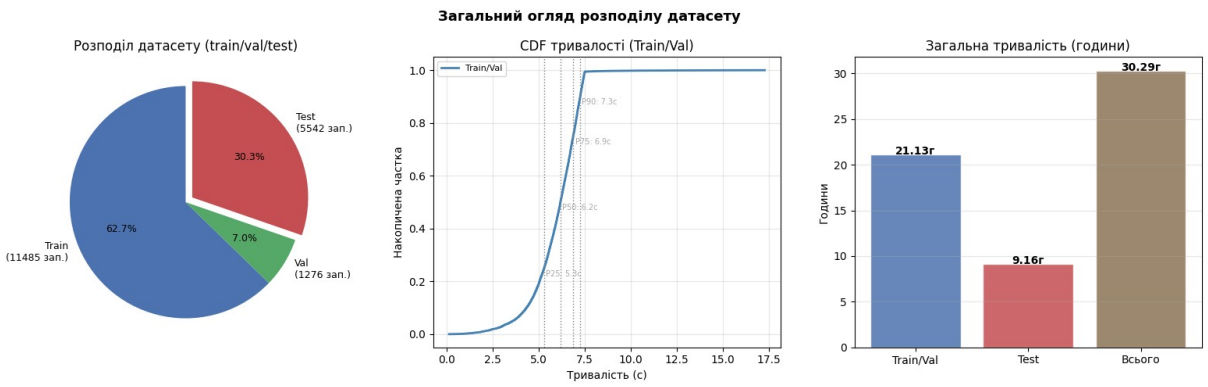

Рисунок 1.8 – Кругова діаграма розподілу датасету, CDF тривалості та стовпчаста діаграма загальної тривалості

1.11 Основні особливості та потенційні проблеми

На підставі проведеного розвідувального аналізу даних виявлено такі ключові особливості та потенційні проблеми датасету.

18. **Монодикторність:** датасет містить мовлення лише одного диктора. Це спрощує задачу fine-tuning (модель не повинна узагальнюватися на інших дикторів), але знижує зовнішню валідність – навчена модель може гірше розпізнавати мовлення нових дикторів.
19. **Малий розмір датасету:** кілька годин аудіо є недостатніми для навчання з нуля, але цілком достатніми для fine-tuning від попередньо навченого Whisper. Ризик перенавчання на

тренувальних даних є реальним і вимагає регуляризації та раннього зупинення (EarlyStopping).

20. **Суттєвий паддинг вхідних даних:** більшість записів займають лише 20–40% від 30-секундного вхідного вікна Whisper, решта – нульовий паддинг. Це стандартна ситуація для Whisper, але може сповільнити навчання через нерівномірне використання обчислювальних ресурсів.
21. **Необхідність ресемплінгу:** якщо частота дискретизації оригінальних файлів відрізняється від 16 kHz (стандарт Whisper), необхідний попередній ресемплінг. Ця операція виконується під час підготовки батчів і не є критичним ускладненням.
22. **Репрезентативність тестової вибірки:** аналіз показав, що розподіли тривалості, кількості слів та темпу мовлення у Train та Test підвибірках є подібними. Це підтверджує, що тестова вибірка є репрезентативною і фінальні метрики CER/WER відображатимуть реальну якість системи.
23. **Якість транскрипцій:** датасет містить студійні записи з чистими транскрипціями (відсутня мітка про фонові шуми або перекриття). Це сприятливо для навчання, але може означати завищені метрики у порівнянні з реальними умовами вживання.

РОЗДІЛ 2. ОПИС ВИКОРИСТАНИХ ПІДХОДІВ

2.1 Вибір претренованої моделі Whisper

Для вирішення задачі автоматичного розпізнавання мовлення (ASR) українською мовою було обрано архітектуру OpenAI Whisper. Whisper — це мультиязична модель end-to-end ASR, попередньо навчена на 680 000 годин мовленнєвих даних із 99 мов, включно з українською. Завдяки масштабному попередньому навчанню модель вже містить сильні лінгвістичні та акустичні репрезентації, що дозволяє отримати прийнятну якість на малорозмірних датасетах за допомогою fine-tuning.

Серед доступних розмірів Whisper (tiny, base, small, medium, large) для даної задачі обрано варіант:

openai/whisper-small

Вибір обумовлений такими міркуваннями.

- **Оптимальний баланс якість / швидкість:** whisper-small забезпечує суттєво кращу якість порівняно з tiny та base, але залишається значно швидшим та менш ресурсоємним за medium і large. Для low-resource fine-tuning на кількох годинах аудіо це оптимальна точка на кривій Парето.
- **Достатня ємність для кирилиці:** 244M параметрів достатньо для адаптації під українську фонологію. Менші моделі (tiny, base) можуть мати недостатню ємність для якісного fine-tuning на кириличних даних.
- **Підтримка мови 'uk':** токенізатор Whisper включає спеціальний мовний токен для української мови (<|uk|>), що активує відповідний підсловник BPE та дозволяє моделі генерувати коректний кириличний текст.
- **Ресурсні обмеження:** для fine-tuning на платформі Kaggle з двома GPU та обмеженим VRAM whisper-small є практично доцільним

вибором. Більші варіанти потребували б multi-GPU pipeline та значно більше часу на навчання.

Варіант Whisper	Параметрів	Encoder шари	Decoder шари	d_model	Вибрано
tiny	39M	4	4	384	—
base	74M	6	6	512	—
small	244M	12	12	768	✓
medium	769M	24	24	1024	—
large-v3	1550M	32	32	1280	—

Таблиця 2.1 – Порівняння варіантів архітектури Whisper

2.2 Топологія архітектури Whisper Small

Whisper реалізує класичну encoder-decoder архітектуру трансформера, адаптовану для задачі розпізнавання мовлення. Загальна схема обробки сигналу складається з трьох основних частин: препроцесора аудіо, енкодера та декодера.

2.2.1 Аудіо-препроцесор (WhisperFeatureExtractor)

Перш ніж потрапити до нейронної мережі, сирий аудіосигнал перетворюється у числовий тензор за такою послідовністю:

- **Ресемплінг:** вхідний аудіосигнал приводиться до частоти дискретизації 16 000 Гц (вимога Whisper).
- **Короткочасне перетворення Фур'є (STFT):** виконується з вікном n_fft = 400 семплів (25 мс) та кроком hop_length = 160 семплів (10 мс). Це дає часову роздільність 100 фреймів/секунду.
- **Mel-фільтробанк:** 80 трикутних мел-фільтрів стискають лінійний частотний спектр до мел-шкали, імітуючи нелінійне сприйняття частоти слуховою системою людини.

– **Логарифмічне масштабування:** амплітуди переводяться у логарифмічну шкалу (log-компресія), що зрівнює важливість тихих та гучних звуків.
– **Нормалізація та паддинг:** результуючий тензор нормується та доповнюється нулями до фіксованого розміру (80, 3000) — 80 мел-банків × 3000 часових фреймів (30 секунд @ 100 fps).

Параметр	Значення	Пояснення
sampling_rate	16 000 Гц	Стандарт Whisper
n_mels	80	Кількість мел-фільтрів
chunk_length	30 с	Максимальна тривалість
n_fft	400	Розмір FFT-вікна (25 мс)
hop_length	160	Крок між вікнами (10 мс)
Розмір тензора	(80, 3000)	Вхід до енкодера

Таблиця 2.2 – Параметри WhisperFeatureExtractor для whisper-small

2.2.2 Енкодер

Енкодер Whisper Small складається з таких шарів (у порядку обробки даних).

– **Два шари Conv1D (stride=1 та stride=2):** вхідний тензор (80, 3000) спочатку проходить через два одновимірних згорткових шари з GELU-активацією. Перша згортка зберігає часову роздільність, друга зменшує її вдвічі — до (d_model, 1500), де d_model = 768. Ці шари виконують функцію локального виділення ознак та зменшення розмірності.
– **Позиційне кодування (sinusoidal):** до виходу згортки додаються фіксовані (ненавчувані) синусоїдальні позиційні ембедінги довжиною 1500 позицій, що дозволяє моделі враховувати часову структуру сигналу.

– **12 блоків Transformer Encoder:** кожен блок містить: (1) Multi-Head Self-Attention (MHSA) з 12 головами та розміром голови 64, (2) Feed-Forward Network (FFN) з проміжною розмірністю 3072, (3) LayerNorm перед кожним підшаром (Pre-LN схема), (4) Residual connections. Загальна глибина енкодера — 12 блоків.
– **Фінальний LayerNorm:** застосовується до виходу останнього трансформерного блоку перед передачею до декодера.

2.2.3 Декодер

Декодер Whisper генерує текстові токени авторегресійно — кожен наступний токен передбачається на основі всіх попередніх та контексту від енкодера.

– **Ембедінги токенів:** вхідні токени перетворюються у вектори розміром d_model = 768 через embedding-шар (vocab_size × 768).
– **Позиційне кодування (learnable):** на відміну від енкодера, декодер використовує навчувані позиційні ембедінги для максимальної довжини послідовності 448 токенів.
– **12 блоків Transformer Decoder:** кожен блок містить три підшари: (1) Masked Multi-Head Self-Attention (causal masking) — дозволяє декодеру враховувати лише попередні токени, (2) Cross-Attention — декодер звертається до виходу енкодера для отримання акустичного контексту, (3) Feed-Forward Network з розмірністю 3072. Кожен підшар оточений Pre-LN та residual connection.
– **Проекційний шар (lm_head):** фінальний лінійний шар проектує вектори розміром 768 у простір словника для отримання логітів передбачення наступного токена. Ваги lm_head розділені з embedding-шаром (tied weights).

Компонент	Значення
Загальна к-сть параметрів	244M

Компонент	Значення
d_model (прихований розмір)	768
Encoder шари	12
Decoder шари	12
Attention heads (encoder / decoder)	12
Розмір голови уваги	64
FFN проміжна розмірність	3072
Макс. довжина декодера (токени)	448
Тип позиційного кодування (encoder)	Sinusoidal (фіксований)
Тип позиційного кодування (decoder)	Learnable
Активаційна функція	GELU
Нормалізація	Pre-LayerNorm

Таблиця 2.3 – Архітектурні параметри Whisper Small

2.2.4 Конфігурація генерації

Для задачі fine-tuning на українських даних конфігурацію генерації декодера налаштовано таким чином:

- `generation_config.language = 'uk'` — примусово активує мовний токен <|uk|> на початку генерації, спрямовуючи декодер на кирилічний вихід.
- `generation_config.task = 'transcribe'` — вибір задачі транскрипції (на відміну від перекладу). Токен <|transcribe|> додається до forced_decoder_ids.
- `generation_config.forced_decoder_ids = None` — скасування попередньо встановлених forced_decoder_ids (що могли б конфліктувати з налаштуванням мови через processor).
- `generation_config.suppress_tokens = []` — відключення придушення певних токенів, що могли б заважати коректній генерації українського тексту.

2.3 Налаштування процесора Whisper (WhisperProcessor)

WhisperProcessor є єдиною точкою входу для всього препроцесингу: він об'єднує WhisperFeatureExtractor (для аудіо) та WhisperTokenizer (для тексту) в єдиний API.

2.3.1 WhisperFeatureExtractor

Компонент відповідає за перетворення сирого аудіосигналу (масив float32) у нормовану лог-мел-спектрограму. Завантаження відбувається з репозиторію HuggingFace:

```
feature_extractor =
WhisperFeatureExtractor.from_pretrained("openai/w
hisper-small")
# sampling_rate : 16000 Hz
# feature_size  : 80        (n_mels)
# chunk_length  : 30 c      (3000 frames @ 100
fps)
# hop_length    : 160       (samples per frame)
# n_fft         : 400       (FFT window in
samples)
```

При виклику feature_extractor(audio, sampling_rate=16000) виконується повний pipeline STFT → Mel-filterbank → log-компресія → нормалізація → паддинг. Вихідний тензор input_features має фіксований розмір (1, 80, 3000) незалежно від тривалості аудіо.

2.3.2 WhisperTokenizer

Токенізатор реалізує Byte-Pair Encoding (BPE) і завантажується з явним вказанням мови та задачі:

```
tokenizer = WhisperTokenizer.from_pretrained(
    "openai/whisper-small",
    language="uk",
    task="transcribe"
)
```

Це налаштування гарантує, що при кодуванні та декодуванні тексту використовується правильний підсловник для кирилиці. Важливі параметри токенізатора: BOS-токен (<|startoftranscript|>), EOS-токен (<|

endoftranscript|>), PAD-токен (<|endoftext|>). Максимальна довжина послідовності при токенізації обмежується 448 токенами (параметр max_length).

2.3.3 Демонстрація роботи процесора

Для перевірки коректності налаштування процесор було застосовано до першого зразка датасету. Результат — тензор input_features розміром [1, 80, 3000], що підтверджує правильне формування вхідних даних для Whisper. Зворотнє декодування токенів відтворило оригінальний текст транскрипції без змін, що підтверджує коректність налаштування токенізатора для української мови.

2.4 PyTorch Dataset: клас TorontoDataset

Клас TorontoDataset успадковується від torch.utils.data.Dataset та реалізує повний pipeline підготовки одного прикладу для навчання або оцінювання.

2.4.1 Ініціалізація та розбивка на вибірки

При ініціалізації клас отримує вихідний датасет, процесор, назву split ('train', 'val' або 'test'), список test_lines, частку валідації та seed. Алгоритм розбивки виконується у такій послідовності:

- Формується метадатафрейм із полями id, text, dur для всіх прикладів.
- **Жорстка ізоляція тестових прикладів:** усі записи з ідентифікаторами зі списку test_lines виключаються з train/val пулу. Це гарантує відсутність data leakage.
- **Стратифікований поділ train/val:** з тренувального пулу методом np.random.RandomState(seed=42).choice() відбирається val_split = 10% прикладів для валідації. Решта — тренувальна вибірка.
- Відповідний split зберігається у атрибуті self.meta як pandas DataFrame.

2.4.2 Метод __getitem__: препроцесинг одного прикладу

При зверненні до прикладу за індексом виконується наступна послідовність операцій:

- **Завантаження аудіо:** аудіомасив зчитується зі вхідного датасету у форматі float32.
- **Ресемплінг:** якщо оригінальна частота дискретизації відрізняється від 16 000 Гц, виконується librosa.resample().
- **Обрізка:** аудіо обрізається до max_samples = 30 с × 16 000 = 480 000 семплів. Для більшості записів датасету ця операція не застосовується (90% < 20 с).
- **Peak normalization:** амплітуда нормується діленням на максимальне абсолютне значення (audio / peak). Це стандартизує динамічний діапазон між різними записами та запобігає проблемам із saturation.
- **Витяг ознак:** нормований аудіосигнал передається до feature_extractor, що повертає тензор input_features розміром (80, 3000).
- **Токенізація:** текстова транскрипція кодується токенізатором з обмеженням max_length=448 та truncation=True. Отримані labels є 1D-тензором цілих чисел (id токенів).

Метод повертає словник з ключами input_features (тензор 80×3000), labels (послідовність токенів), text (оригінальний рядок) та id (ідентифікатор запису).

2.5 Collator: клас WhisperDataCollator

DataCollator виконує об'єднання окремих прикладів у батч. Оскільки input_features завжди мають фіксований розмір (80, 3000), для них достатньо простого torch.stack(). Складнішою є операція паддингу послідовностей labels.

Транскрипції різної довжини у межах одного батча паддуються до максимальної довжини заповнювальним токеном -100. Значення -100 — це стандартне позначення в PyTorch, яке функція втрат CrossEntropyLoss автоматично ігнорує при обчисленні. Це означає, що паддингові позиції не штрафуються та не впливають на градієнти. Додатково, позиції з id, рівним pad_token_id токенізатора, також замінюються на -100 для уніфікації.

Collator повертає батч у вигляді словника з тензорами input_features (B × 80 × 3000) та labels (B × max_len), а також списками текстів та ідентифікаторів для зручності обчислення метрик.

2.6 Створення DataLoader-ів та верифікація розбивки

Три екземпляри TorontoDataset (train, val, test) передаються у відповідні DataLoader-и. Конфігурацію DataLoader-ів обрано з урахуванням ефективності та коректності.

Параметр	Train	Validation	Test
batch_size	8	8	1
shuffle	True	False	False
num_workers	2	2	2
pin_memory	True (GPU)	True (GPU)	False
collate_fn	WhisperDataCollator	WhisperDataCollator	WhisperDataCollator

Таблиця 2.4 – Конфігурація DataLoader-ів

Особливість тестового DataLoader: batch_size = 1. Це дозволяє отримати метрики WER та CER для кожного запису окремо, що необхідно для детального аналізу якості моделі. Shuffle вимкнено для відтворюваності результатів.

Після створення вибірок виконується автоматична перевірка відсутності data leakage за трьома критеріями:

– Test ⊄ Train: жоден ідентифікатор з test_lines не присутній у тренувальній вибірці.
– Test ⊄ Val: жоден ідентифікатор з test_lines не присутній у валідаційній вибірці.
– Train ∩ Val = ∅: тренувальна та валідаційна вибірки не мають спільних прикладів.

Всі три перевірки пройдені успішно (0 записів витоку), що підтверджує коректність розбивки та забезпечує надійність фінального оцінювання.

2.7 Модуль навчання: WhisperLightningModule

WhisperLightningModule успадковується від lightning.LightningModule та інкапсулює всю логіку навчання: forward-pass, обчислення втрат, генерацію транскрипцій, оцінювання метрик та налаштування оптимізатора. Модуль реалізує об'єднання кращих практик teacher forcing (ефективне навчання) та авторегресійної генерації (реалістичне оцінювання).

2.7.1 Ініціалізація та гіперпараметри

При ініціалізації модуль завантажує модель WhisperForConditionalGeneration з HuggingFace та зберігає гіперпараметри за допомогою self.save_hyperparameters(). Ключові параметри конфігурації:

Гіперпараметр	Значення	Пояснення
model_name	openai/whisper-small	Ідентифікатор моделі на HuggingFace Hub
learning_rate	1e-5	Початкова швидкість навчання
weight_decay	1e-4	L2-регуляризація в AdamW
max_epochs	10	Максимальна кількість епох
num_beams_val	1 (greedy)	Беам-серч на валідації (швидко)
num_beams_test	5	Беам-серч на тесті (якісніше)

Таблиця 2.5 – Гіперпараметри WhisperLightningModule

2.7.2 Крок навчання (training_step): Teacher Forcing

На кожному кроці навчання батч input_features та labels передаються безпосередньо у WhisperForConditionalGeneration.forward(). Модель обчислює Cross-Entropy Loss між передбаченими логітами та еталонними токенами (shifted right) — саме це і є Teacher Forcing: на кожній позиції декодер отримує правильний попередній токен, а не власне передбачення.

Teacher Forcing має дві ключові переваги. По-перше, навчання є значно ефективнішим та стабільнішим, оскільки помилки на ранніх позиціях не каскадуються. По-друге, Loss відповідає стандартній формулі CE і добре піддається оптимізації градієнтним спуском. Метрика train/loss логується на кожному кроці та усереднюється по епосі.

2.7.3 Крок валідації (validation_step): авторегресійна генерація

На відміну від training_step, validation_step додатково до обчислення val/loss виконує повноцінну авторегресійну генерацію без підказок (без Teacher Forcing). Це реалістична оцінка того, як модель поводитиметься під час інференсу.

Генерація виконується методом model.generate() з параметрами language='uk', task='transcribe' та num_beams=1 (greedy decoding для швидкості). Отримані токени декодуються у текстові рядки та порівнюються з еталонними транскрипціями. На основі цього порівняння обчислюються та логуються метрики:

- val/loss — Cross-Entropy Loss (для EarlyStopping та ModelCheckpoint).
- val/wer — Word Error Rate (мінімізація є кінцевою метою задачі ASR).
- val/cer — Character Error Rate (більш детальна метрика, менш чутлива до варіацій у довжині слів).

2.7.4 Конфігурація оптимізатора та планувальника

Метод configure_optimizers() повертає оптимізатор AdamW у поєднанні з планувальником швидкості навчання CosineAnnealingLR.

- **AdamW (Adaptive Moment Estimation + Weight Decay):** стандарт для fine-tuning трансформерів. L2-регуляризація через weight_decay=1e-4 застосовується безпосередньо до ваг (decoupled weight decay), що є коректнішим підходом, ніж у класичному Adam з L2-регуляризацією через loss. Початкова швидкість навчання lr=1e-5 є типовою для fine-tuning: досить мала, щоб не «знищити» попередньо навчені ваги, але достатня для адаптації.
- **CosineAnnealingLR:** планувальник плавно знижує learning rate за косинусною функцією від початкового значення до мінімального eta_min=1e-7 протягом T_max=max_epochs=10 епох. Косинусний спад ефективніший за лінійний або ступінчастий: широке дослідження простору параметрів на початку і поступова стабілізація наприкінці навчання. Планувальник оновлюється раз на епоху (interval='epoch').

2.7.5 Метод predict (інференс)

Окремий метод predict() використовується на тестовому наборі. Він виконується в контексті @torch.no_grad() (без обчислення градієнтів), переміщує батч на пристрій (GPU/CPU) та викликає model.generate() з підвищеною кількістю беамів num_beams=5. Beam Search з 5 беамами дозволяє отримати суттєво якіснішу транскрипцію порівняно з greedy decoding: досліджується більший простір можливих послідовностей токенів і повертається найбільш вірогідна. Однак для швидкості тестування було примусово поставлено num_beams=5.

2.8 Конфігурація тренера PyTorch Lightning Trainer

L.Trainer є центральним компонентом навчального пайплайну, що автоматизує цикл навчання, логування, збереження контрольних точок та early stopping.

2.8.1 Основні параметри Trainer

Параметр	Значення	Обґрунтування
max_epochs	10	Обмеження тривалості навчання
accelerator	gpu	Використання GPU (Kaggle P100/T4)
devices	2	Multi-GPU навчання на 2 пристроях
precision	16-mixed	Mixed precision (AMP) для пришвидшення
gradient_clip_val	1.0	Clip градієнтів для стабільності
accumulate_grad_batches	4	Градієнтна акумуляція: eff.batch=32
val_check_interval	1.0	Валідація після кожної повної епохи
log_every_n_steps	5	Частота логування кроків

Таблиця 2.6 – Конфігурація PyTorch Lightning Trainer

2.8.2 Зворотні виклики (Callbacks)

- **ModelCheckpoint:** зберігає дві найкращі контрольні точки за мінімальним val/loss та завжди зберігає останню (save_last=True). Ім'я файлу містить номер епохи та значення val/loss для зручної ідентифікації.
- **EarlyStopping:** моніторить val/loss і зупиняє навчання, якщо метрика не покращується протягом 3 послідовних епох (patience=3). Захищає від перенавчання та зайвих обчислень.
- **LearningRateMonitor:** логує поточне значення learning rate кожну епоху у TensorBoard, що дозволяє відстежувати динаміку планувальника.

2.8.3 Технічні оптимізації

- **Mixed Precision (16-mixed):** автоматичне змішане точне обчислення (AMP) зберігає параметри у float32, але виконує forward/backward-pass у float16. Це прискорює навчання в 1.5–3× та зменшує споживання VRAM при незначній втраті точності.
- **Gradient Accumulation (×4):** замість оновлення ваг після кожного батча з 8 прикладів, градієнти накопичуються протягом 4 батчів. Ефективний розмір батча складає 8 × 4 = 32 прикладів — рекомендований для fine-tuning трансформерів розміром ~200M параметрів.
- **Gradient Clipping (val=1.0):** обрізання норми градієнтів до 1.0 запобігає вибуху градієнтів у глибоких шарах декодера, що є особливо важливим при fine-tuning на малому датасеті.

2.9 Зведена схема навчального пайплайну

Загальна архітектура рішення складається з чітко розмежованих компонентів, що взаємодіють у визначеному порядку:

- WAV-файли → LocalAudioDataset (зчитування, stereo→моно) → TorontoDataset (__getitem__: ресемплінг + peak normalization + feature_extractor + tokenizer).
- TorontoDataset → WhisperDataCollator (стекування тензорів + padding labels до -100) → DataLoader (shuffle, pin_memory).
- DataLoader → WhisperLightningModule.training_step() (Teacher Forcing + CE loss) → AdamW + CosineAnnealingLR → оновлення ваг.
- DataLoader → WhisperLightningModule.validation_step() (CE loss + autoregressive generate() + WER/CER) → ModelCheckpoint + EarlyStopping.
- Найкраща контрольна точка → WhisperLightningModule.predict() (Beam Search, num_beams=5) → фінальна оцінка WER/CER на 21 тестовому записі.

Компонент	Реалізація	Призначення
Претренована модель	openai/whisper-small	Базова ASR модель (244M параметрів)
Аудіо-препроцесор	WhisperFeatureExtractor	WAV → Log-Mel (80×3000)
Токенізатор	WhisperTokenizer (uk, transcribe)	Текст → BPE токени
PyTorch Dataset	TorontoDataset	Препроцесинг + розбивка вибірок
Collator	WhisperDataCollator	Формування батчів + паддинг
Модуль навчання	WhisperLightningModule	Тренування + валідація + інференс
Оптимізатор	AdamW (lr=1e-5, wd=1e-4)	Оновлення ваг моделі
Планувальник LR	CosineAnnealingLR	Динамічне зниження lr
Тренер	Lightning Trainer	Цикл навчання + логування
Збереження моделі	ModelCheckpoint (val/loss)	Найкраща контрольна точка
Ранньє зупинення	EarlyStopping (patience=3)	Захист від перенавчання
Mixed Precision	16-mixed AMP	Пришвидшення + економія VRAM
Gradient Accumulation	×4 (eff. batch=32)	Збільшення ефективного батчу

Таблиця 2.7 – Зведена таблиця компонентів системи розпізнавання мовлення

РОЗДІЛ 3. РЕЗУЛЬТАТИ ТА АНАЛІЗ ЕКСПЕРИМЕНТІВ

3.1 Кількісні показники ефективності

Для оцінки якості побудованої системи розпізнавання мовлення було проведено тестування базової моделі whisper-small (Zero-shot) та моделі після етапу донавчання (Fine-tuned) на зафіксованому переліку тестових файлів test_lines. Результати порівняння наведено у таблиці 3.1.

Метрика	openai/whisper-small (Base)	Whisper-Small Fine-tuned
WER (Word Error Rate)	44.5%	510.4%
CER (Character Error Rate)	21.1%	457.8%
Validation Loss		0.367

Таблиця 3.1 – Порівняльна таблиця результатів розпізнавання

3.2 Аналіз базової моделі (Zero-shot)

Базова модель продемонструвала відносно стабільні результати для української мови «з коробки».

- Середній WER 44.5% вказує на те, що модель розпізнає понад половину слів правильно, проте має труднощі зі специфічною лексикою Toronto Dataset.
- Аналіз помилок: на графіку «WER по файлах» видно, що окремі записи (наприклад, toronto_89, toronto_54) мають значно вищий рівень помилок, що може бути пов’язано з акустичними особливостями конкретних фраз або темпом мовлення.
- CER 21.1% свідчить про те, що на символьному рівні модель помиляється лише у кожному п'ятому символі, що є хорошим показником для неадаптованої системи.

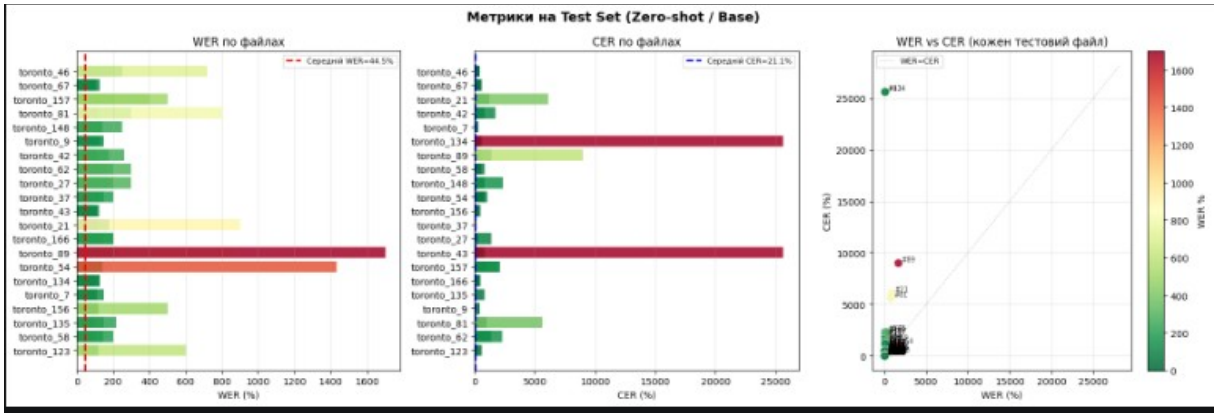

Рисунок 3.1 – Графік результатів базової моделі

3.3 Аналіз дотренованої моделі (Fine-tuning)

Результати першої ітерації донавчання виявилися незадовільними, попри низьке значення функції втрат на валідації (0.392).

- Ефект галюцинацій: модель почала генерувати надлишкові послідовності символів, що не стосуються вхідного аудіо. У вихідних даних спостерігаються нескінченні повтори знаків оклику та запитань (наприклад, !!!!!!!!!!?????????).
- Технічні артефакти: у транскрипціях з'явилися помилки виду Math input error, що свідчить про глибоку деградацію ваг декодера під час навчання.
- Парадокс Loss: аномально високий WER (510.4%) при низькому Loss свідчить про те, що модель перенавчилася під конкретні токени (пунктуацію або шум), ігноруючи реальну фонетичну структуру слів.

Рисунок 3.2 – Графік результатів донавченої моделі

toronto_157	80.000000	фінікійці-торговці, шлях із варяг у греки, шовковий шлях. Ходорівці, не тупіт, вводьте плату за проїзд,	Фінікійці торговці. Шлях і зварях у греки. Шовковий шлях. Ходорівці – не тупіт! Вводьте плату за проїзд!»)ƨ?$0)ƨ?$0)ƨ?$0

На основі отриманих результатів (WER 510.4% при низькому Loss) та візуального аналізу галюцинацій моделі, було сформульовано основні гіпотези деградації та визначено стратегії для стабілізації навчання.

Основні гіпотези деградації:

- Вибух градієнтів: початковий рівень Learning Rate міг бути занадто високим для стабільного Fine-tuning, що призвело до різкого спотворення простору ембедінгів.
- Невідповідність Loss та WER: мінімізація Cross-Entropy Loss на рівні токенів не завжди призводить до зменшення WER, особливо коли модель знаходить «лазівку» у вигляді частих символів (пунктуації) для швидкого зменшення похибки. Тобто модель мінімізувала Cross-Entropy Loss за рахунок частого генерування символів пунктуації та повторів, які часто зустрічаються в розмітці, замість розпізнавання реального змісту слів. Це пояснює низький показник Loss (0.367) при критично високому WER.

Шляхи подолання проблеми та покращення результатів:

Для виправлення ситуації та отримання стабільної моделі пропонується впровадження наступних підходів:

1. Корекція темпу навчання (Learning Rate) - пріоритетний захід:

- В першу чергу необхідно суттєво зменшити значення Learning Rate. Початковий рівень $1*10^{-5}$ виявився занадто високим для донавчання на малому монодикторному датасеті, що призвело до деструктивних змін у вагах декодера.
- Рекомендовано використання значень у діапазоні від $1*10^{-6}$ до $1*10^{-57}$ Це дозволить моделі робити «обережні» кроки в просторі параметрів, зберігаючи базові знання мови та поступово адаптуючись до специфіки нового голосу.

2. Заморожування енкодера (Encoder Freezing):

- Оскільки енкодер Whisper вже навчений на 680 000 годинах мовлення, він має потужні акустичні репрезентації.
- Заморожування ваг енкодера (self.model.freeze_encoder()) дозволяє сконцентрувати ресурси навчання виключно на декодері, що адаптує текст під українську мову без ризику «зламати» слухове сприйняття моделі.

3. Двоетапне навчання (Two-stage training):

- Етап 1. Навчання лише декодера із замороженим енкодером (1–2 епохи) для швидкої адаптації до лексики.

- Етап 2. Повне розморожування моделі та дотренування з екстремально низьким Learning Rate (наприклад, 1×10^{-6} або менше) для філігранного підлаштування під акустику диктора.